

🌟 The Master Guide to Data Structures & Algorithms

A Comprehensive Reference for Programmers, Engineers & Interview Prep

📖 Table of Contents

1. **Foundations & Analysis**
 - Big-O, Big-Omega, Big-Theta Notation
 - Recursion vs. Iteration & Memory Models
2. **Sorting Algorithms**
 - Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix, Bucket, Timsort
3. **Searching & Pattern Matching**
 - Linear, Binary, Exponential, Ternary Search
 - KMP, Rabin-Karp, Aho-Corasick
4. **Graph Algorithms**
 - BFS, DFS, Dijkstra, Bellman-Ford, Floyd-Warshall, A*, Kruskal, Prim, Topological Sort, Tarjan's SCC
5. **Dynamic Programming & Greedy Algorithms**
 - Fractional Knapsack, Huffman Coding, Activity Selection
 - 0/1 Knapsack, LCS, LIS, Matrix Chain Multiplication, Edit Distance, Kadane's
6. **Advanced Data Structures**
 - BST, AVL, Red-Black Trees, Segment Tree, Fenwick Tree, Trie, Union-Find
7. **Special & Modern Topics**
 - Backtracking (N-Queens, Sudoku), Bit Manipulation, Reservoir Sampling, Bloom Filters

💡 **How to use this guide:** Each algorithm follows the same structure — intuition, complexity, real-world use, working Python code, and pitfalls — so you can jump to any section and get everything you need.

① Foundations & Analysis

🔗 Big-O, Big-Omega, Big-Theta Notation

🔗 Intuitive Explanation

Complexity notation describes how an algorithm's running time or memory use **grows** as input size (n) grows — not the exact time, but the shape of the growth curve.

- **Big-O (O)** — the **upper bound** ("worst case won't be worse than this"). This is what people mean 95% of the time they say "Big-O."
- **Big-Omega (Ω)** — the **lower bound** ("best case won't be better than this").
- **Big-Theta (Θ)** — a **tight bound** — the algorithm grows at *exactly* this rate, both upper and lower.

Think of it like a speed limit range for a trip: Ω is the fastest you could possibly arrive, O is the slowest, and Θ is when both meet — you know exactly how long it takes regardless of traffic.

📊 Common Complexity Classes (fastest → slowest)

Notation	Name	Example
$O(1)$	Constant	Array index lookup
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Single loop through a list
$O(n \log n)$	Linearithmic	Merge sort, quicksort (avg)
$O(n^2)$	Quadratic	Nested loops (bubble sort)
$O(2^n)$	Exponential	Naive recursive Fibonacci
$O(n!)$	Factorial	Brute-force permutations

🔧 Code: Measuring Growth Empirically

```
import time

def linear_search(nums, target):
    for i, n in enumerate(nums):
        if n == target:
            return i
    return -1

# Demonstrate O(n) growth: doubling input roughly doubles time
for size in [10_000, 20_000, 40_000]:
    data = list(range(size))
    start = time.perf_counter()
    linear_search(data, -1) # worst case: not found, scans
    # everything
    elapsed = time.perf_counter() - start
    print(f"n={size:>7}: {elapsed:.5f}s")
```

⚠️ Pitfalls

- Big-O ignores constants: $O(100n)$ is still " $O(n)$," but for small n a "worse" algorithm can run faster in practice.
- Always ask "Big-O of what?" — time AND space complexity matter, and they can differ.
- Average-case complexity often matters more than worst-case in production (e.g., Quicksort is $O(n^2)$ worst case but $O(n \log n)$ average, and is usually preferred over Heap Sort for this reason).

🔄 Recursion vs. Iteration & Memory Models

💡 Intuitive Explanation

Iteration repeats a block of code using a loop, reusing the same memory each pass. **Recursion** solves a problem by having a function call itself on a smaller version of the problem, relying on the **call stack** to remember where to "return to."

Every recursive call pushes a new **stack frame** (local variables, return address) onto the call stack. Too many nested calls without a base case → **stack overflow**.

🕒 Time & Space Complexity

Approach	Time		Space
Iterative sum of n items	$O(n)$	$O(1)$	
Recursive sum of n items	$O(n)$	$O(n)$	— one stack frame per call

🌐 Real-World Use Case

Recursion is the natural fit for anything **self-similar**: file system traversal (folders contain folders), parsing nested JSON/HTML/XML, tree and graph traversal, and divide-and-conquer algorithms (merge sort). Iteration is preferred in performance-critical paths and when recursion depth could be large (Python's default recursion limit is ~1000).

🔗 Code: Same Problem, Both Ways

```
# Iterative factorial - O(1) space
def factorial_iter(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# Recursive factorial - O(n) space (call stack)
def factorial_rec(n):
    if n <= 1:                # base case
        return 1
    return n * factorial_rec(n - 1)

# Tail-call style (Python doesn't optimize this, but shown for
concept)
def factorial_tail(n, accumulator=1):
    if n <= 1:
        return accumulator
    return factorial_tail(n - 1, n * accumulator)

print(factorial_iter(10), factorial_rec(10), factorial_tail(10)) #
3628800 x3
```

⚠️ Edge Cases & Pitfalls

- Python has **no tail-call optimization** — a “tail-recursive” function still uses $O(n)$ stack space, unlike in Scheme or Haskell.
- Missing or unreachable base case → infinite recursion → `RecursionError`.
- Deep recursion (e.g., processing a 100,000-node linked list recursively) will crash in Python; convert to iteration or raise `sys.setrecursionlimit()` cautiously.

2 Sorting Algorithms

★ Bubble Sort — Comparison Sort

🔍 How It Works

Repeatedly step through the list, compare adjacent items, and swap them if they're in the wrong order. Each full pass “bubbles” the largest remaining item to the end. Repeat until no swaps are needed.

🕒 Complexity

Case	Time	Space
Best (already sorted)	$O(n)$	$O(1)$
Average	$O(n^2)$	$O(1)$
Worst	$O(n^2)$	$O(1)$

🌐 Real-World Use

Rarely used in production due to poor performance — but its simplicity makes it a teaching tool, and it's occasionally used for **nearly-sorted small datasets** where the early-exit optimization pays off (e.g., re-sorting a small list after one item changes).

🔗 Code

```
def bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:      # early exit: already sorted
            break
    return arr

print(bubble_sort([5, 1, 4, 2, 8])) # [1, 2, 4, 5, 8]
```

⚠️ When NOT to Use

Never use for large datasets ($n > \sim 1,000$) — $O(n^2)$ becomes painfully slow. Use Timsort (Python's built-in `sorted()`) instead in virtually all real code.

★ Selection Sort — *Comparison Sort*

🔗 How It Works

Divide the list into a sorted and unsorted region. Repeatedly find the **minimum** element in the unsorted region and move it to the end of the sorted region.

🕒 Complexity

Case	Time	Space
Best	$O(n^2)$	$O(1)$
Average	$O(n^2)$	$O(1)$
Worst	$O(n^2)$	$O(1)$

🌐 Real-World Use

Useful when **writes are expensive** (e.g., writing to flash memory) since it makes at most $O(n)$ swaps, unlike bubble/insertion sort which can make $O(n^2)$ swaps.

🔗 Code

```
def selection_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```

arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr

print(selection_sort([64, 25, 12, 22, 11])) # [11, 12, 22, 25, 64]

```

⚠ When NOT to Use

Not stable by default (equal elements can be reordered), and always $O(n^2)$ even on sorted input — worse than bubble sort's best case. Avoid for large or performance-sensitive datasets.

★ Insertion Sort — *Comparison Sort*

🔧 How It Works

Build the sorted list one item at a time — take each new item and insert it into its correct position among the already-sorted items to its left, similar to how you'd sort playing cards in your hand.

⌚ Complexity

Case	Time	Space
Best (nearly sorted)	$O(n)$	$O(1)$
Average	$O(n^2)$	$O(1)$
Worst (reverse sorted)	$O(n^2)$	$O(1)$

🌐 Real-World Use

Excellent for **small or nearly-sorted arrays**. Timsort (used by Python and Java) actually switches to insertion sort for small sub-arrays (< 64 elements) because it's faster than merge sort at that scale.

💻 Code

```

def insertion_sort(arr):
    arr = arr.copy()
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([12, 11, 13, 5, 6])) # [5, 6, 11, 12, 13]

```

⚠ When NOT to Use

Poor choice for large, randomly-ordered datasets — $O(n^2)$ average case. Great for streaming data where you insert one new item into an already-sorted collection.

★ Merge Sort — *Divide and Conquer*

🔧 How It Works

1. **Divide** the list in half recursively until each sublist has 1 element (trivially sorted).
2. **Conquer**: merge sorted sublists back together, always taking the smaller “front” element from each side.

⚙ Complexity

Case	Time	Space
Best	$O(n \log n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

🌐 Real-World Use

Guaranteed $O(n \log n)$ makes it ideal when predictable performance matters — external sorting (sorting data too large for RAM, merging sorted chunks from disk), and it’s **stable**, which matters for multi-key sorts (e.g., sort by last name, then by first name).

💻 Code

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return _merge(left, right)

def _merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:      # <= keeps it stable
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10])) # [3, 9, 10, 27, 38, 43, 82]
```

⚠ When NOT to Use

Uses $O(n)$ extra memory — not ideal for memory-constrained environments (embedded systems) where in-place Quick Sort or Heap Sort is preferable.

★ Quick Sort — *Divide and Conquer*

⚙ How It Works

1. Pick a **pivot** element.
2. **Partition**: rearrange the array so everything smaller than the pivot is to its left, everything larger is to its right.
3. Recursively sort the left and right partitions.

⚙ Complexity

Case	Time	Space
Best	$O(n \log n)$	$O(\log n)$
Average	$O(n \log n)$	$O(\log n)$
Worst (bad pivot choices)	$O(n^2)$	$O(n)$

🌐 Real-World Use

The **default general-purpose sort** in many C library implementations (qsort). Great cache performance (sorts in-place) makes it faster in practice than merge sort for in-memory arrays despite the same average complexity class.

🔗 Code

```
import random

def quick_sort(arr):
    arr = arr.copy()
    _quick_sort(arr, 0, len(arr) - 1)
    return arr

def _quick_sort(arr, low, high):
    if low < high:
        pivot_idx = _partition(arr, low, high)
        _quick_sort(arr, low, pivot_idx - 1)
        _quick_sort(arr, pivot_idx + 1, high)

def _partition(arr, low, high):
    # random pivot avoids worst-case  $O(n^2)$  on already-sorted input
    rand_idx = random.randint(low, high)
    arr[rand_idx], arr[high] = arr[high], arr[rand_idx]
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

print(quick_sort([10, 7, 8, 9, 1, 5])) # [1, 5, 7, 8, 9, 10]
```

⚠️ Edge Cases & Pitfalls

- **Always-worst-case trap:** picking the first or last element as pivot on an already-sorted array degrades to $O(n^2)$. Random or median-of-three pivot selection avoids this.
- Not stable by default.
- Deep recursion on adversarial input can hit Python's recursion limit.

★ Heap Sort — Comparison Sort (Selection-based)

🔗 How It Works

1. Build a **max-heap** from the array (largest element at the root).
2. Repeatedly swap the root (max) with the last element, shrink the heap by one, and “sift down” to restore the heap property.

🕒 Complexity

Case	Time	Space
Best	$O(n \log n)$	$O(1)$
Average	$O(n \log n)$	$O(1)$
Worst	$O(n \log n)$	$O(1)$

🌐 Real-World Use

Guarantees $O(n \log n)$ **worst case** with $O(1)$ extra space — used when memory is tight and predictable performance matters (embedded/real-time systems), and it's the algorithm behind **priority queues**.

🔗 Code

```
def heap_sort(arr):
    arr = arr.copy()
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1): # build max heap
        _sift_down(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0] # move max to the end
        _sift_down(arr, i, 0)
    return arr

def _sift_down(arr, size, root):
    largest = root
    left, right = 2 * root + 1, 2 * root + 2
    if left < size and arr[left] > arr[largest]:
        largest = left
    if right < size and arr[right] > arr[largest]:
        largest = right
    if largest != root:
        arr[root], arr[largest] = arr[largest], arr[root]
        _sift_down(arr, size, largest)

print(heap_sort([12, 11, 13, 5, 6, 7])) # [5, 6, 7, 11, 12, 13]
```

⚠️ When NOT to Use

Not stable, and poor cache locality (jumps around the array) makes it slower **in practice** than Quick Sort despite the same Big-O — prefer Quick Sort unless you specifically need the worst-case guarantee.

✂️ Counting Sort — *Non-Comparison Sort*

🔍 How It Works

Count how many times each value appears (in a range of known size k), then reconstruct the sorted array directly from those counts — no comparisons at all.

⚙️ Complexity

Case	Time	Space
All cases	$O(n + k)$	$O(n + k)$

k = range of input values.

🌐 Real-World Use

Sorting **exam scores (0-100)**, sorting pixel intensity values (0-255) in image processing, or any dataset with a small, known integer range.

🔗 Code

```
def counting_sort(arr):
    if not arr:
        return []
    min_val, max_val = min(arr), max(arr)
    k = max_val - min_val + 1
    counts = [0] * k
    for n in arr:
        counts[n - min_val] += 1
    result = []
    for i, count in enumerate(counts):
        result.extend([i + min_val] * count)
    return result

print(counting_sort([4, 2, 2, 8, 3, 3, 1])) # [1, 2, 2, 3, 3, 4, 8]
```

⚠ When NOT to Use

Terrible for large value ranges ($k \gg n$) — sorting `[1, 1_000_000]` wastes huge memory on a counts array. Only use when the value range is small and known.

★ Radix Sort — *Non-Comparison Sort*

⚙ How It Works

Sort integers digit by digit, from least significant to most significant, using a stable sort (usually counting sort) as a subroutine at each digit position.

⌚ Complexity

Case	Time	Space
All cases	$O(d \times (n + k))$	$O(n + k)$

d = number of digits, k = base (usually 10).

🌐 Real-World Use

Sorting large sets of **fixed-length keys**: phone numbers, IP addresses, employee IDs, or postal codes — anywhere keys have bounded digit length regardless of how large n is.

🔗 Code

```
def radix_sort(arr):
    if not arr:
        return []
    max_val = max(arr)
    exp = 1
    arr = arr.copy()
    while max_val // exp > 0:
        arr = _counting_sort_by_digit(arr, exp)
        exp *= 10
    return arr

def _counting_sort_by_digit(arr, exp):
    n = len(arr)
```

```

output = [0] * n
counts = [0] * 10
for num in arr:
    digit = (num // exp) % 10
    counts[digit] += 1
for i in range(1, 10):
    counts[i] += counts[i - 1]
for i in range(n - 1, -1, -1):           # iterate backward for
stability
    digit = (arr[i] // exp) % 10
    counts[digit] -= 1
    output[counts[digit]] = arr[i]
return output

print(radix_sort([170, 45, 75, 90, 802, 24, 2, 66])) # [2, 24, 45,
66, 75, 90, 170, 802]

```

⚠ When NOT to Use

Only works cleanly on non-negative integers (or fixed-format strings) — needs adaptation for negative numbers or floats. Not a comparison sort, so it can't be used as a drop-in replacement when a custom comparator is required.

✳ Bucket Sort — *Distribution Sort*

🔧 How It Works

Distribute elements into a number of **buckets** based on value range, sort each bucket individually (usually with insertion sort), then concatenate the buckets in order.

⌚ Complexity

Case	Time	Space
Best (uniform distribution)	$O(n + k)$	$O(n + k)$
Average	$O(n + k)$	$O(n + k)$
Worst (all in one bucket)	$O(n^2)$	$O(n + k)$

🌐 Real-World Use

Sorting **uniformly distributed floating-point data**, such as normalized scores (0.0–1.0) or grading curves.

🔗 Code

```

def bucket_sort(arr, bucket_count=10):
    if not arr:
        return []
    min_val, max_val = min(arr), max(arr)
    range_size = (max_val - min_val) / bucket_count or 1
    buckets = [[] for _ in range(bucket_count)]
    for n in arr:
        idx = min(int((n - min_val) / range_size), bucket_count - 1)
        buckets[idx].append(n)
    result = []
    for bucket in buckets:
        result.extend(sorted(bucket))           # insertion sort in
practice for small buckets
    return result

print(bucket_sort([0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21]))

```

⚠ When NOT to Use

Degrades to $O(n^2)$ if data is clustered (all values land in one bucket) rather than uniformly spread — know your data distribution before choosing this.

★ Timsort — *Hybrid Sort (Merge + Insertion)*

🔧 How It Works

Python's and Java's built-in sort. It finds naturally occurring sorted "runs" in the data, extends short runs using insertion sort (up to ~64 elements), then merges runs together using merge sort's merging logic — exploiting real-world data's partial order.

🕒 Complexity

Case	Time	Space
Best (already sorted)	$O(n)$	$O(n)$
Average	$O(n \log n)$	$O(n)$
Worst	$O(n \log n)$	$O(n)$

🌐 Real-World Use

This is what runs every time you call `sorted()` or `list.sort()` in Python, or `Arrays.sort()` on objects in Java. It's the sort behind spreadsheet column sorting, database ORDER BY on in-memory result sets, and virtually all everyday application sorting.

💻 Code

```
# You don't reimplement Timsort in practice — you use it directly:
data = [5, 2, 9, 1, 5, 6]
print(sorted(data))           # [1, 2, 5, 5, 6, 9]

# Custom key/comparator support is Timsort's real power in real
apps:
employees = [{"name": "Ana", "age": 34}, {"name": "Sam", "age": 28}]
print(sorted(employees, key=lambda e: e["age"]))

# Stability matters here: sorting by age preserves original relative
# order of employees who share the same age.
```

⚠ Edge Cases & Pitfalls

- Since it's stable and adaptive, don't reach for a hand-rolled sort "for performance" in Python — Timsort (implemented in C) will almost always beat a hand-written Python sort.
 - Custom key functions are called once per element ($O(n)$), not per comparison — prefer `key=` over `cmp=`-style comparators for performance.
-

③ Searching & Pattern Matching

★ Linear Search — *Sequential Search*

🔍 How It Works

Check every element one by one until the target is found or the list ends.

⚙️ Complexity

Case	Time	Space
Best	$O(1)$	$O(1)$
Average	$O(n)$	$O(1)$
Worst	$O(n)$	$O(1)$

🌐 Real-World Use

Searching **unsorted data** — scanning a small unindexed list, or any dataset too small or too dynamic to justify sorting first.

💻 Code

```
def linear_search(nums, target):
    for i, n in enumerate(nums):
        if n == target:
            return i
    return -1

print(linear_search([4, 2, 7, 1], 7)) # 2
```

⚠️ When NOT to Use

Never use on large, sortable, frequently-searched data — sort once and use binary search instead.

★ Binary Search — *Divide and Conquer Search*

🔍 How It Works

On a **sorted** list, repeatedly check the middle element and eliminate the half that can't contain the target.

⚙️ Complexity

Case	Time	Space
Best	$O(1)$	$O(1)$
Average	$O(\log n)$	$O(1)$
Worst	$O(\log n)$	$O(1)$

🌐 Real-World Use

Database indexing (B-tree lookups use this principle), `git bisect` for finding the commit that introduced a bug, autocomplete/dictionary lookups, and any “find the boundary” problem (e.g., first version of an API that supports a feature).

💻 Code

```
def binary_search(nums, target):
    low, high = 0, len(nums) - 1
```

```

while low <= high:
    mid = (low + high) // 2
    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return -1

print(binary_search([1, 3, 5, 7, 9, 11], 7)) # 3

```

⚠ Edge Cases & Pitfalls

- **Requires sorted data** — running it on unsorted data gives silently wrong answers, not an error.
- Classic bug: `mid = (low + high) // 2` can overflow in languages with fixed-size integers (not an issue in Python, but critical in Java/C++ — use `low + (high - low) // 2`).

★ Exponential Search — *Search on Unbounded/Sorted Data*

🔍 How It Works

Find a range where the target could exist by repeatedly doubling an index (1, 2, 4, 8, 16...) until you overshoot the target, then run binary search within that range.

⚙ Complexity

Case	Time	Space
Best	$O(1)$	$O(1)$
Average	$O(\log n)$	$O(1)$
Worst	$O(\log n)$	$O(1)$

🌐 Real-World Use

Searching in **unbounded or streaming sorted data** where you don't know the size in advance — e.g., searching an infinite sorted stream, or searching sorted data structures where “size” is expensive to compute.

💻 Code

```

def exponential_search(nums, target):
    if not nums:
        return -1
    if nums[0] == target:
        return 0
    i = 1
    while i < len(nums) and nums[i] <= target:
        i *= 2
    low = i // 2
    high = min(i, len(nums) - 1)
    return _binary_search_range(nums, target, low, high)

def _binary_search_range(nums, target, low, high):
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:

```

```

        return mid
    elif nums[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
    return -1

print(exponential_search([1, 3, 5, 7, 9, 11, 13, 15], 11)) # 5

```

⚠ When NOT to Use

For normal, bounded, in-memory arrays, plain binary search is simpler and just as fast — exponential search shines specifically when the upper bound is unknown.

★ Ternary Search — *Divide into Three*

🔍 How It Works

Similar to binary search but splits the range into **three** parts using two midpoints, discarding one-third of the range each time. Also used to find the peak/minimum of a **unimodal function** (one that strictly increases then strictly decreases, or vice versa).

⌚ Complexity

Case	Time	Space
All cases (sorted array search)	$O(\log_3 n)$	$O(1)$

Note: this is still $O(\log n)$ — the base doesn't matter asymptotically, and ternary search actually does **more comparisons** than binary search in practice for simple lookups.

🌐 Real-World Use

Finding the **maximum/minimum of a unimodal function** — used in optimization problems, such as finding the optimal price point on a concave revenue curve, or tuning a single parameter in machine learning where the loss function is convex in that parameter.

💻 Code

```

def ternary_search_max(f, low, high, epsilon=1e-9):
    """Find the x that maximizes unimodal function f in [low,
    high]."""
    while high - low > epsilon:
        m1 = low + (high - low) / 3
        m2 = high - (high - low) / 3
        if f(m1) < f(m2):
            low = m1
        else:
            high = m2
    return (low + high) / 2

# Example: find the peak of -(x-2)^2 + 10 (a downward parabola
# peaking at x=2)
peak_x = ternary_search_max(lambda x: -(x - 2) ** 2 + 10, low=-10,
high=10)
print(round(peak_x, 2)) # ~2.0

```

⚠ When NOT to Use

For simple sorted-array lookups, binary search is strictly better (fewer comparisons per step). Ternary search's real value is **function optimization**, not array search — don't confuse the two use cases.

★ Knuth-Morris-Pratt (KMP) — *String Pattern Matching*

🔧 How It Works

Naive string search re-checks characters it's already seen after a mismatch. KMP precomputes a “**failure function**” (LPS array — Longest Proper Prefix which is also Suffix) for the pattern, so on a mismatch it knows exactly how far it can safely skip ahead **without** re-scanning the text.

⌚ Complexity

Case	Time	Space
All cases	$O(n + m)$	$O(m)$

n = text length, m = pattern length.

🌐 Real-World Use

Text editors' “find” feature, DNA sequence matching in bioinformatics, and network intrusion detection systems scanning packet payloads for known attack signatures.

💻 Code

```
def kmp_search(text, pattern):
    if not pattern:
        return []
    lps = _build_lps(pattern)
    matches = []
    i = j = 0 # i -> text index, j -> pattern index
    while i < len(text):
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                matches.append(i - j) # match found
                j = lps[j - 1]
        else:
            if j != 0:
                j = lps[j - 1] # skip using failure
            else:
                i += 1
    return matches

def _build_lps(pattern):
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length != 0:
            length = lps[length - 1]
```

```

        else:
            lps[i] = 0
            i += 1
        return lps

print(kmp_search("ababcbabababc", "ababc")) # [7]

```

⚠ When NOT to Use

Overkill for one-off searches on short strings — Python's built-in `str.find()` (which uses a highly optimized algorithm) is faster in practice for most cases. KMP's value shows up when searching **repeatedly** or on **very large texts**.

★ Rabin-Karp — *Hash-Based String Matching*

🔧 How It Works

Compute a **rolling hash** of the pattern and of each substring of the text with the same length. Compare hashes first (cheap); only do a full character comparison if hashes match (to rule out hash collisions).

⌚ Complexity

Case	Time	Space
Best/Average	$O(n + m)$	$O(1)$
Worst (many hash collisions)	$O(n \times m)$	$O(1)$

🌐 Real-World Use

Plagiarism detection and **duplicate document detection** — since Rabin-Karp naturally extends to searching for **multiple patterns at once** by hashing them all first, it's used in systems that check submitted text against a large corpus.

💻 Code

```

def rabin_karp_search(text, pattern, base=256, prime=101):
    n, m = len(text), len(pattern)
    if m > n:
        return []
    pattern_hash = 0
    window_hash = 0
    h = pow(base, m - 1, prime)
    matches = []

    for i in range(m):
        pattern_hash = (base * pattern_hash + ord(pattern[i])) % prime
        window_hash = (base * window_hash + ord(text[i])) % prime

    for i in range(n - m + 1):
        if pattern_hash == window_hash:           # cheap check
            if text[i:i + m] == pattern:         # confirm to
                matches.append(i)                 avoid false positive
        if i < n - m:
            window_hash = (base * (window_hash - ord(text[i]) * h) +
                           ord(text[i + m])) % prime

```

```

        window_hash = (window_hash + prime) % prime # keep
positive
        return matches

print(rabin_karp_search("abcbabc", "cab")) # [2, 5]

```

⚠ Edge Cases & Pitfalls

- **Hash collisions** cause false positive candidates — always verify with a direct string comparison (shown above), or worst-case degrades to $O(n \times m)$.
- Choosing a poor prime/base can increase collisions; a large prime is recommended for production use.

✳ Aho-Corasick — *Multi-Pattern String Matching*

🔧 How It Works

Builds a **Trie** of all patterns to search for simultaneously, then adds **failure links** (similar to KMP's failure function) so that on a mismatch, the automaton jumps to the longest possible suffix match instead of restarting — finding **all patterns in one pass** over the text.

⌚ Complexity

Case	Time	Space
Build automaton	$O(\Sigma \text{ pattern lengths})$	$O(\Sigma \text{ pattern lengths} \times \text{alphabet size})$
Search	$O(n + \text{total matches})$	$O(1)$ additional

🌐 Real-World Use

Antivirus/malware scanners checking a file against thousands of known virus signatures in one pass, network intrusion detection (matching against many attack patterns simultaneously), and content filtering systems (matching against large blocklists of words/phrases).

📄 Code

```

from collections import deque, defaultdict

class AhoCorasick:
    def __init__(self, patterns):
        self.goto = [defaultdict(int)] # trie transitions
        self.fail = [0] # failure links
        self.output = [[]] # patterns ending at
each node

        for pattern in patterns:
            self._add_pattern(pattern)
            self._build_failure_links()

    def _add_pattern(self, pattern):
        node = 0
        for ch in pattern:
            if ch not in self.goto[node]:
                self.goto[node].append(defaultdict(int))
                self.fail.append(0)
                self.output.append([])
            self.goto[node][ch] = len(self.goto) - 1

```

```

        node = self.goto[node][ch]
        self.output[node].append(pattern)

    def _build_failure_links(self):
        queue = deque()
        for ch, node in self.goto[0].items():
            self.fail[node] = 0
            queue.append(node)
        while queue:
            current = queue.popleft()
            for ch, nxt in self.goto[current].items():
                queue.append(nxt)
                fallback = self.fail[current]
                while fallback and ch not in self.goto[fallback]:
                    fallback = self.fail[fallback]
                self.fail[nxt] = self.goto[fallback].get(ch, 0) if
self.goto[fallback].get(ch, 0) != nxt else 0
                self.output[nxt] += self.output[self.fail[nxt]]

    def search(self, text):
        node = 0
        results = []
        for i, ch in enumerate(text):
            while node and ch not in self.goto[node]:
                node = self.fail[node]
            node = self.goto[node].get(ch, 0)
            for pattern in self.output[node]:
                results.append((i - len(pattern) + 1, pattern))
        return results

ac = AhoCorasick(["he", "she", "his", "hers"])
print(ac.search("ushers")) # [(1, 'she'), (2, 'he'), (2, 'hers')]

```

⚠ When NOT to Use

Significant setup cost (building the automaton) means it's only worthwhile when searching for **many patterns** repeatedly. For a single pattern, use KMP; for a handful of patterns searched once, simple iteration may be simpler to maintain.

④ Graph Algorithms

★ Breadth-First Search (BFS) — *Level-by-Level Traversal*

🔍 How It Works

Explore a graph **level by level** using a queue: visit all neighbors of the start node first, then all neighbors of those neighbors, and so on.

⌚ Complexity

Case	Time	Space
All cases	$O(V + E)$	$O(V)$

V = vertices, E = edges.

🌐 Real-World Use

Shortest path in unweighted graphs — social network “degrees of separation” (LinkedIn’s “how you’re connected”), web crawlers, and finding the shortest route in a maze/grid game.

🔗 Code

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

graph = {"A": ["B", "C"], "B": ["A", "D"], "C": ["A", "D"], "D": ["B", "C"]}
print(bfs(graph, "A")) # ['A', 'B', 'C', 'D']
```

⚠ When NOT to Use

Uses more memory than DFS for wide/shallow graphs (stores an entire “frontier” level in the queue). Not suitable for weighted shortest-path problems — use Dijkstra’s instead.

★ Depth-First Search (DFS) — *Deep-Dive Traversal*

🔗 How It Works

Explore as far as possible along each branch before backtracking, using a stack (or recursion, which uses the call stack implicitly).

⌚ Complexity

Case	Time	Space
All cases	$O(V + E)$	$O(V)$

🌐 Real-World Use

Cycle detection, solving mazes, topological sorting (build systems, task scheduling), and detecting connected components (e.g., “which users belong to the same friend cluster”).

🔗 Code

```
def dfs(graph, start, visited=None, order=None):
    if visited is None:
        visited = set()
        order = []
    visited.add(start)
    order.append(start)
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)
```

```

        return order

graph = {"A": ["B", "C"], "B": ["A", "D"], "C": ["A", "D"], "D":
["B", "C"]}
print(dfs(graph, "A")) # ['A', 'B', 'D', 'C']

```

⚠ Edge Cases & Pitfalls

- Recursive DFS can hit Python’s recursion limit on very deep/large graphs — use an explicit stack for large graphs.
- Doesn’t find the **shortest** path in unweighted graphs (BFS does) — DFS just finds *a* path.

✳ Dijkstra’s Algorithm — *Shortest Path, Non-Negative Weights*

🔍 How It Works

Maintain a running “best known distance” to every node. Repeatedly pick the **unvisited node with the smallest known distance** (using a min-heap), and relax (update) the distances to its neighbors.

⌚ Complexity

Case	Time (with min-heap)	Space
All cases	$O((V + E) \log V)$	$O(V)$

🌐 Real-World Use

GPS navigation (finding the fastest route), network routing protocols (OSPF), and flight-price search engines finding the cheapest connection between airports.

📄 Code

```

import heapq

def dijkstra(graph, start):
    """graph: {node: [(neighbor, weight), ...]}"""
    distances = {node: float("inf") for node in graph}
    distances[start] = 0
    heap = [(0, start)]
    while heap:
        current_dist, node = heapq.heappop(heap)
        if current_dist > distances[node]:
            continue # stale entry, skip
        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight
            if new_dist < distances[neighbor]:
                distances[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return distances

graph = {
    "A": [("B", 4), ("C", 1)],
    "B": [("D", 1)],
    "C": [("B", 2), ("D", 5)],
    "D": [],
}
print(dijkstra(graph, "A")) # {'A': 0, 'B': 3, 'C': 1, 'D': 4}

```

⚠ When NOT to Use

Fails with negative edge weights (can give wrong answers, doesn't detect the issue) — use Bellman-Ford instead. Also unnecessary overkill for unweighted graphs, where plain BFS is simpler and just as correct.

★ Bellman-Ford Algorithm — *Shortest Path with Negative Weights*

🔧 How It Works

Relax **every edge**, $V - 1$ times. Because the shortest path between any two nodes visits at most $V - 1$ edges, this guarantees convergence. A final extra pass can detect **negative-weight cycles**.

⌚ Complexity

Case	Time	Space
All cases	$O(V \times E)$	$O(V)$

🌐 Real-World Use

Currency arbitrage detection (finding a cycle of currency exchanges that yields “free money,” modeled as a negative-weight cycle), and network routing protocols like RIP that must handle “cost” edges that can effectively be negative in some formulations.

💻 Code

```
def bellman_ford(edges, vertices, start):
    """edges: list of (u, v, weight)"""
    distances = {v: float("inf") for v in vertices}
    distances[start] = 0

    for _ in range(len(vertices) - 1):
        for u, v, weight in edges:
            if distances[u] != float("inf") and distances[u] +
weight < distances[v]:
                distances[v] = distances[u] + weight

    # Detect negative-weight cycles
    for u, v, weight in edges:
        if distances[u] != float("inf") and distances[u] + weight <
distances[v]:
            raise ValueError("Graph contains a negative-weight
cycle")

    return distances

edges = [("A", "B", 4), ("A", "C", 1), ("C", "B", 2), ("B", "D", 1),
("C", "D", 5)]
vertices = ["A", "B", "C", "D"]
print(bellman_ford(edges, vertices, "A")) # {'A': 0, 'B': 3, 'C':
1, 'D': 4}
```

⚠ When NOT to Use

Significantly slower than Dijkstra's ($O(V \times E)$ vs $O((V+E) \log V)$) — only use it when negative edge weights are actually possible in your data.

✦ Floyd-Warshall Algorithm — *All-Pairs Shortest Path*

🔧 How It Works

Dynamic programming: for every pair of nodes (i, j), check if going **through** an intermediate node k gives a shorter path than the current known distance. Try every k as a possible intermediate stop.

⚙️ Complexity

Case	Time	Space
All cases	$O(V^3)$	$O(V^2)$

🌐 Real-World Use

Precomputing **all shortest distances** in a road network for a delivery service that needs fast repeated lookups, and finding the “transitive closure” of relationships in small networks.

💻 Code

```
def floyd_warshall(graph_matrix):
    """graph_matrix[i][j] = weight, or float('inf') if no direct
    edge."""
    n = len(graph_matrix)
    dist = [row.copy() for row in graph_matrix]

    for k in range(n):
        for i in range(n):
            for j in range(n):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]

    return dist

INF = float("inf")
graph_matrix = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0],
]
result = floyd_warshall(graph_matrix)
for row in result:
    print(row)
```

⚠️ When NOT to Use

$O(V^3)$ makes it impractical for large graphs ($V > \sim 500$) — for a single source, use Dijkstra’s or Bellman-Ford instead; only use Floyd-Warshall when you need **every** pair’s distance.

✦ A* Search — *Heuristic-Guided Shortest Path*

🔧 How It Works

Like Dijkstra’s, but prioritizes nodes using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost so far and $h(n)$ is a **heuristic estimate** of remaining cost to the goal (e.g., straight-line distance). A good

heuristic dramatically reduces how much of the graph is explored.

🔍 Complexity

Case	Time	Space
Best (perfect heuristic)	$O(\text{path length})$	$O(V)$
Worst (poor heuristic)	$O((V + E) \log V)$	$O(V)$

🌐 Real-World Use

Video game pathfinding (NPCs navigating a map), GPS routing with real-world heuristics (straight-line distance to destination), and robotics motion planning.

🔗 Code

```
import heapq

def a_star(graph, start, goal, heuristic):
    """graph: {node: [(neighbor, weight), ...]}, heuristic: {node:
    estimated_cost_to_goal}"""
    open_set = [(heuristic[start], start)]
    g_score = {start: 0}
    came_from = {}

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            path = [current]
            while current in came_from:
                current = came_from[current]
            path.append(current)
            return path[::-1], g_score[goal]

        for neighbor, weight in graph[current]:
            tentative_g = g_score[current] + weight
            if tentative_g < g_score.get(neighbor, float("inf")):
                came_from[neighbor] = current
                g_score[neighbor] = tentative_g
                f_score = tentative_g + heuristic[neighbor]
                heapq.heappush(open_set, (f_score, neighbor))

    return None, float("inf") # no path found

graph = {"A": [("B", 1), ("C", 4)], "B": [("D", 2)], "C": [("D",
1)], "D": []}
heuristic = {"A": 3, "B": 2, "C": 1, "D": 0}
print(a_star(graph, "A", "D", heuristic)) # (['A', 'B', 'D'], 3)
```

⚠️ Edge Cases & Pitfalls

- The heuristic must be **admissible** (never overestimate the true cost) or A* can return a suboptimal path.
- With $h(n) = 0$ everywhere, A* becomes plain Dijkstra's — the heuristic quality is what gives A* its speed advantage.

★ Kruskal's Algorithm — Minimum Spanning Tree (Edge-based)

🔍 How It Works

Sort all edges by weight. Add each edge to the MST **as long as it doesn't create a cycle** (checked using Union-Find), until all nodes are connected.

⌚ Complexity

Case	Time	Space
All cases	$O(E \log E)$	$O(V)$

🌐 Real-World Use

Designing **minimum-cost network infrastructure** — laying the least total cable to connect a set of buildings, or designing a road network connecting cities with minimum total construction cost.

💻 Code

```
def kruskal(vertices, edges):
    """edges: list of (weight, u, v)"""
    parent = {v: v for v in vertices}

    def find(v):
        while parent[v] != v:
            parent[v] = parent[parent[v]] # path compression
            v = parent[v]
        return v

    def union(u, v):
        root_u, root_v = find(u), find(v)
        if root_u == root_v:
            return False
        parent[root_u] = root_v
        return True

    mst = []
    for weight, u, v in sorted(edges):
        if union(u, v):
            mst.append((u, v, weight))
    return mst

vertices = ["A", "B", "C", "D"]
edges = [(1, "A", "B"), (3, "B", "C"), (4, "A", "C"), (2, "C", "D")]
print(kruskal(vertices, edges)) # [('A', 'B', 1), ('C', 'D', 2), ('B', 'C', 3)]
```

⚠ When NOT to Use

On **dense** graphs (many edges), Prim's with a good priority queue can be faster since Kruskal's sorting step is $O(E \log E)$ regardless of density.

★ Prim's Algorithm — Minimum Spanning Tree (Vertex-based)

🔍 How It Works

Start from any node. Repeatedly add the **cheapest edge** that connects a node already in the MST to a node not yet in it — grows the tree one vertex at a time, always via a min-heap.

⌚ Complexity

Case	Time (with min-heap)	Space
All cases	$O(E \log V)$	$O(V)$

🌐 Real-World Use

Same domain as Kruskal's (network design), but preferred for **dense graphs** — e.g., designing a circuit board layout minimizing wire length between components.

🔧 Code

```
import heapq

def prim(graph, start):
    """graph: {node: [(neighbor, weight), ...]}"""
    visited = {start}
    edges = [(weight, start, neighbor) for neighbor, weight in
graph[start]]
    heapq.heapify(edges)
    mst = []

    while edges and len(visited) < len(graph):
        weight, u, v = heapq.heappop(edges)
        if v in visited:
            continue
        visited.add(v)
        mst.append((u, v, weight))
        for neighbor, w in graph[v]:
            if neighbor not in visited:
                heapq.heappush(edges, (w, v, neighbor))
    return mst

graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("A", 1), ("C", 3), ("D", 2)],
    "C": [("A", 4), ("B", 3), ("D", 5)],
    "D": [("B", 2), ("C", 5)],
}
print(prim(graph, "A")) # [('A','B',1), ('B','D',2), ('B','C',3)]
```

⚠ When NOT to Use

Requires the whole graph to be connected — doesn't naturally handle disconnected graphs (Kruskal's does, producing a minimum spanning **forest** instead).

✳ Topological Sort — *Dependency Ordering (DAGs)*

🔧 How It Works

Order the nodes of a **Directed Acyclic Graph (DAG)** so that every edge $u \rightarrow v$ has u appearing before v . Typically done via DFS (append to result on "finish," then reverse) or Kahn's algorithm (repeatedly remove nodes with in-degree 0).

🕒 Complexity

Case	Time	Space
All cases	$O(V + E)$	$O(V)$

🌐 Real-World Use

Build systems (compiling files in dependency order), **task scheduling** (must finish task A before task B), package manager dependency resolution (npm install, pip install), and course prerequisite planning.

🔗 Code

```
from collections import deque

def topological_sort(graph):
    """Kahn's algorithm. graph: {node: [dependents...]}"""
    in_degree = {node: 0 for node in graph}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    queue = deque([n for n in graph if in_degree[n] == 0])
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    if len(order) != len(graph):
        raise ValueError("Graph has a cycle – no valid topological order")

    return order

graph = {"shirt": ["jacket"], "pants": ["shoes", "jacket"], "shoes": [], "jacket": []}
print(topological_sort(graph)) # e.g. ['shirt', 'pants', 'shoes', 'jacket']
```

⚠️ Edge Cases & Pitfalls

- Only defined for **DAGs** — a cycle means no valid ordering exists (the code above detects and raises on this).
- Multiple valid topological orders often exist; don't assume a unique "correct" answer.

★ Tarjan's Algorithm — Strongly Connected Components

🔗 How It Works

A single DFS pass that tracks, for each node, a **discovery time** and the **lowest discovery time reachable** from it (low-link value). Nodes form a Strongly Connected Component (SCC) when a node's low-link equals its own discovery time, using a stack to pop off the current SCC.

🕒 Complexity

Case	Time	Space
All cases	$O(V + E)$	$O(V)$

🌐 Real-World Use

Detecting circular dependencies in software modules, analyzing the web graph for tightly-interlinked clusters of pages, and finding groups of mutually reachable states in a state machine (useful in compiler optimization for dead-code and loop detection).

🔧 Code

```
def tarjan_scc(graph):
    index_counter = [0]
    stack = []
    lowlink = {}
    index = {}
    on_stack = {}
    result = []

    def strongconnect(node):
        index[node] = index_counter[0]
        lowlink[node] = index_counter[0]
        index_counter[0] += 1
        stack.append(node)
        on_stack[node] = True

        for neighbor in graph[node]:
            if neighbor not in index:
                strongconnect(neighbor)
            lowlink[node] = min(lowlink[node],
                                lowlink[neighbor])

        elif on_stack.get(neighbor):
            lowlink[node] = min(lowlink[node], index[neighbor])

        if lowlink[node] == index[node]:
            scc = []
            while True:
                w = stack.pop()
                on_stack[w] = False
                scc.append(w)
                if w == node:
                    break
            result.append(scc)

    for node in graph:
        if node not in index:
            strongconnect(node)
    return result

graph = {"A": ["B"], "B": ["C"], "C": ["A", "D"], "D": ["E"], "E":
["D"]}
print(tarjan_scc(graph)) # [['E', 'D'], ['A', 'C', 'B']] (order may
vary)
```

⚠️ When NOT to Use

Recursive implementation can hit stack limits on very large/deep graphs — an iterative version is preferable for production use on graphs with millions of nodes.

⑤ Dynamic Programming & Greedy Algorithms

✦ Fractional Knapsack — Greedy

🔧 How It Works

Given items with weight and value, and a knapsack of limited capacity, you may take **fractions** of items. Greedy strategy: sort items by **value-to-weight ratio**, descending, and take as much as possible of the best ratio first.

⚙️ Complexity

Case	Time	Space
All cases (dominated by sort)	$O(n \log n)$	$O(1)$

🌐 Real-World Use

Resource allocation where partial use is possible — cutting inventory to fill an order, allocating fractional CPU/bandwidth to maximize utility, blending ingredients to maximize value within a budget.

💻 Code

```
def fractional_knapsack(items, capacity):
    """items: list of (value, weight)"""
    items = sorted(items, key=lambda x: x[0] / x[1], reverse=True)
    total_value = 0.0
    for value, weight in items:
        if capacity <= 0:
            break
        take = min(weight, capacity)
        total_value += take * (value / weight)
        capacity -= take
    return total_value

items = [(60, 10), (100, 20), (120, 30)] # (value, weight)
print(fractional_knapsack(items, capacity=50)) # 240.0
```

⚠️ When NOT to Use

Doesn't apply if items are **indivisible** (can't take 0.6 of a physical item) — that's the 0/1 Knapsack problem, which requires DP, not greedy.

★ Huffman Coding — Greedy, Optimal Prefix Codes

🔧 How It Works

Build a binary tree bottom-up: repeatedly take the two **least frequent** symbols and merge them into a new node (frequency = sum), until one tree remains. Frequent symbols end up with short codes, rare symbols with long codes — minimizing total encoded length.

⚙️ Complexity

Case	Time	Space
All cases	$O(n \log n)$	$O(n)$

🌐 Real-World Use

Lossless file compression — the core idea behind ZIP, JPEG’s entropy coding stage, and MP3 compression.

🔗 Code

```
import heapq
from collections import Counter

class Node:
    def __init__(self, freq, char=None, left=None, right=None):
        self.freq, self.char, self.left, self.right = freq, char,
left, right
    def __lt__(self, other):
        return self.freq < other.freq

def huffman_encoding(text):
    freq = Counter(text)
    heap = [Node(f, ch) for ch, f in freq.items()]
    heapq.heapify(heap)

    while len(heap) > 1:
        left = heapq.heappop(heap)
        right = heapq.heappop(heap)
        heapq.heappush(heap, Node(left.freq + right.freq, left=left,
right=right))

    root = heap[0]
    codes = {}
    def assign_codes(node, code=""):
        if node.char is not None:
            codes[node.char] = code or "0" # handle single-char
edge case
        return
        assign_codes(node.left, code + "0")
        assign_codes(node.right, code + "1")
    assign_codes(root)
    return codes

codes = huffman_encoding("aaabbc")
print(codes) # e.g. {'a': '0', 'b': '11', 'c': '10'}
```

⚠ Edge Cases & Pitfalls

- Requires **prior knowledge of frequencies** — for streaming data with unknown distribution, adaptive Huffman coding is needed instead.
- A text with only one unique character needs a special case (shown above) since a single-node tree has no natural “0/1” path.

★ Activity Selection — Greedy Interval Scheduling

🔗 How It Works

Given activities with start/end times, select the **maximum number of non-overlapping** activities. Greedy strategy: sort by **end time**, and always pick the next activity whose start time is $\geq$ the end time of the last selected one.

🕒 Complexity

Case	Time	Space
All cases (dominated by sort)	$O(n \log n)$	

🌐 Real-World Use

Meeting room scheduling (maximize meetings in one room), CPU job scheduling to maximize throughput, and classroom/lecture-hall booking systems.

🔗 Code

```
def activity_selection(activities):
    """activities: list of (start, end)"""
    activities = sorted(activities, key=lambda a: a[1])  # sort by
end time
    selected = [activities[0]]
    last_end = activities[0][1]

    for start, end in activities[1:]:
        if start >= last_end:
            selected.append((start, end))
            last_end = end
    return selected

activities = [(1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6,
10), (8, 11)]
print(activity_selection(activities))  # [(1,4), (5,7), (8,11)]
```

⚠️ When NOT to Use

This greedy approach only guarantees optimality for the **count-maximizing** version. If activities have different **values/weights** and you want to maximize total value (not just count), you need **Weighted Interval Scheduling**, solved with DP instead.

✳️ 0/1 Knapsack — *Dynamic Programming*

🔗 How It Works

Unlike fractional knapsack, each item must be taken **whole or not at all**. Build a DP table $dp[i][w]$ = max value using the first i items with capacity w . For each item, choose the better of “skip it” vs “take it” (if it fits).

⌚ Complexity

Case	Time	Space
All cases	$O(n \times W)$	$O(n \times W)$, reducible to $O(W)$

n = items, W = capacity.

🌐 Real-World Use

Budget allocation (choosing which indivisible projects to fund within a fixed budget to maximize ROI), cargo loading (whole packages only), and investment portfolio selection with a fixed capital limit.

🔗 Code

```
def knapsack_01(weights, values, capacity):
    n = len(weights)
```

```

dp = [[0] * (capacity + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    for w in range(capacity + 1):
        dp[i][w] = dp[i - 1][w] # don't
take item i
        if weights[i - 1] <= w:
            dp[i][w] = max(dp[i][w],
                           dp[i - 1][w - weights[i - 1]] +
values[i - 1]) # take it
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
print(knapsack_01(weights, values, capacity=7)) # 9 (items with
weight 3+4)

```

⚠ When NOT to Use

$O(n \times W)$ is **pseudo-polynomial** — if capacity W is enormous (e.g., billions), this becomes impractical even for small n . Approximation algorithms are used at that scale.

★ Longest Common Subsequence (LCS) — Dynamic Programming

🔍 How It Works

Find the longest sequence that appears (not necessarily contiguously) in both strings, in order. Build a DP table where $dp[i][j]$ = LCS length of the first i characters of string A and first j of string B.

⌚ Complexity

Case	Time	Space
All cases	$O(m \times n)$	$O(m \times n)$, reducible to $O(\min(m,n))$

🌐 Real-World Use

diff tools and version control (Git computing what changed between file versions), DNA sequence alignment in bioinformatics, and plagiarism/similarity detection between documents.

💻 Code

```

def lcs(text_a, text_b):
    m, n = len(text_a), len(text_b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text_a[i - 1] == text_b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

print(lcs("ABCBADAB", "BDCABA")) # 4 ("BCBA" or "BDAB")

```

⚠ When NOT to Use

For very long strings (e.g., whole-genome comparison), $O(m \times n)$ space/time is prohibitive — specialized algorithms (Hirschberg's, suffix automata) are used for large-scale bioinformatics.

★ Longest Increasing Subsequence (LIS) — *Dynamic Programming*

🔧 How It Works

Find the longest subsequence where each element is strictly greater than the previous. Naive DP: $dp[i]$ = length of the longest increasing subsequence ending at index i . An optimized version uses **binary search** with a “tails” array to reach $O(n \log n)$.

⚙️ Complexity

Case	Time	Space
Naive DP	$O(n^2)$	$O(n)$
Optimized (binary search)	$O(n \log n)$	$O(n)$

🌐 Real-World Use

Stock price analysis (longest streak of increasing prices), scheduling problems (finding the longest chain of compatible tasks by increasing deadline), and patience-sorting-based algorithms.

💻 Code

```
import bisect

def lis_length(nums):
    if not nums:
        return 0
    tails = []
    for n in nums:
        pos = bisect.bisect_left(tails, n)
        if pos == len(tails):
            tails.append(n)
        else:
            tails[pos] = n
    return len(tails)

print(lis_length([10, 9, 2, 5, 3, 7, 101, 18])) # 4 ([2,3,7,101]
or [2,3,7,18])
```

⚠️ Edge Cases & Pitfalls

- The tails array in the optimized version is **not** the actual LIS itself, just tracks lengths — reconstructing the actual sequence needs extra bookkeeping (parent pointers).
- Watch for “strictly increasing” vs “non-decreasing” — the comparison (`bisect_left` vs `bisect_right`) changes based on which the problem requires.

★ Matrix Chain Multiplication — *Dynamic Programming*

🔧 How It Works

Given a chain of matrices to multiply, find the **order of multiplication (parenthesization)** that minimizes the total number of scalar multiplications — the result is the same regardless of order, but the cost isn't. Build a DP table over all sub-chains.

⌚ Complexity

Case	Time	Space
All cases	$O(n^3)$	$O(n^2)$

🌐 Real-World Use

Computer graphics and scientific computing — optimizing the order of matrix transformations in a 3D rendering pipeline or numerical simulation, where the matrices themselves can be huge and multiplication order drastically affects performance.

💻 Code

```
def matrix_chain_order(dims):
    """dims: dimensions where matrix i has shape dims[i-1] x
    dims[i]"""
    n = len(dims) - 1 # number of matrices
    dp = [[0] * n for _ in range(n)]

    for length in range(2, n + 1): # chain length
        for i in range(n - length + 1):
            j = i + length - 1
            dp[i][j] = float("inf")
            for k in range(i, j):
                cost = (dp[i][k] + dp[k + 1][j] +
                        dims[i] * dims[k + 1] * dims[j + 1])
                dp[i][j] = min(dp[i][j], cost)
    return dp[0][n - 1]

dims = [40, 20, 30, 10, 30] # 4 matrices
print(matrix_chain_order(dims)) # 26000 (minimum scalar
multiplications)
```

⚠️ When NOT to Use

Only relevant when matrices vary significantly in size — if all matrices are similarly sized (e.g., square, same dimensions), the ordering barely matters and this optimization isn't worth the complexity.

★ Edit Distance (Levenshtein Distance) — Dynamic Programming

🔍 How It Works

Find the minimum number of insertions, deletions, or substitutions to transform one string into another. $dp[i][j]$ = edit distance between the first i chars of A and first j chars of B — built from the cost of the three possible operations.

⌚ Complexity

Case	Time	Space
All cases	$O(m \times n)$	$O(m \times n)$, reducible to $O(\min(m,n))$

🌐 Real-World Use

Spell checkers and autocorrect (how close is your typo to a real word), DNA sequence similarity, and fuzzy search matching (e.g., “did you mean...?” suggestions).

🔗 Code

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(m + 1):
        dp[i][0] = i                # delete all of a
    for j in range(n + 1):
        dp[0][j] = j                # insert all of b

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]    # no operation
            else:
                dp[i][j] = 1 + min(
                    dp[i - 1][j],    # delete
                    dp[i][j - 1],    # insert
                    dp[i - 1][j - 1], # substitute
                )
    return dp[m][n]

print(edit_distance("kitten", "sitting")) # 3
```

⚠ When NOT to Use

For very long strings, $O(m \times n)$ is expensive — approximate/banded variants exist for near-matches when strings are known to be similar in length and content.

★ Kadane’s Algorithm — Dynamic Programming (Maximum Subarray)

🔗 How It Works

Find the contiguous subarray with the **largest sum**. Track a running sum; whenever it drops below the current element itself, it’s better to **restart** the subarray from here rather than carry negative baggage forward.

🕒 Complexity

Case	Time	Space
All cases	$O(n)$	$O(1)$

🌐 Real-World Use

Stock trading analysis (best contiguous period of gains), signal processing (finding the strongest contiguous burst in sensor data), and image processing (maximum brightness region in a 1D scan line).

🔗 Code

```
def kadane_max_subarray(nums):
    max_ending_here = max_so_far = nums[0]
    for n in nums[1:]:
        max_ending_here = max(n, max_ending_here + n)
        max_so_far = max(max_so_far, max_ending_here)
    return max_so_far

print(kadane_max_subarray([-2, 1, -3, 4, -1, 2, 1, -5, 4])) # 6
([4, -1, 2, 1])
```

⚠ Edge Cases & Pitfalls

- If the list is **all negative**, the algorithm correctly returns the least-negative single element — don't special-case it to return 0 unless the problem explicitly allows an "empty" subarray.
- This is the classic example of recognizing that **local optimal choice** (restart vs. extend) leads to a global optimum — a hallmark of DP/greedy hybrid thinking.

⑥ Advanced Data Structures & Tree Algorithms

★ Binary Search Tree (BST) — *Ordered Tree Structure*

🔧 How It Works

A tree where, for every node, all values in its **left** subtree are smaller and all values in its **right** subtree are larger. This ordering allows insert, search, and delete in $O(\log n)$ *on average*.

⌚ Complexity

Operation	Best/Average	Worst (unbalanced)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

🌐 Real-World Use

In-memory ordered maps/sets where you need fast lookup **and** ordered traversal (e.g., range queries: "give me all users aged 25-35"). Basis for many language library structures (Java's `TreeMap`, C++'s `std::map`).

📝 Code

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
```

```

        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        elif val > node.val:
            node.right = self._insert(node.right, val)
        return node

    def search(self, val):
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None or node.val == val:
            return node is not None
        return self._search(node.left, val) if val < node.val else
self._search(node.right, val)

    def inorder(self):
        result = []
        def _walk(node):
            if node:
                _walk(node.left)
                result.append(node.val)
                _walk(node.right)
        _walk(self.root)
        return result

bst = BST()
for v in [5, 3, 7, 1, 4, 6, 8]:
    bst.insert(v)
print(bst.inorder())      # [1, 3, 4, 5, 6, 7, 8] - sorted!
print(bst.search(6))      # True

```

⚠ When NOT to Use

A BST built from **already-sorted data** degrades to a linked list ($O(n)$ operations) — always use a **self-balancing** variant (AVL, Red-Black) when insertion order isn't random or guaranteed shuffled.

★ AVL Tree — Self-Balancing BST (Strict Balance)

🔧 How It Works

An AVL Tree is a BST that automatically **rotates** nodes after insertions/deletions to keep the height difference between left and right subtrees (the “balance factor”) at most 1 — guaranteeing $O(\log n)$ height at all times.

⌚ Complexity

Operation	All cases (guaranteed)
Search / Insert / Delete	$O(\log n)$

🌐 Real-World Use

Systems needing **guaranteed** fast lookups regardless of insertion pattern, such as database indexing where worst-case latency matters (real-time systems), and in-memory databases requiring predictable

performance.

🔗 Code

```
class AVLNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.height = 1

def height(node):
    return node.height if node else 0

def balance_factor(node):
    return height(node.left) - height(node.right) if node else 0

def update_height(node):
    node.height = 1 + max(height(node.left), height(node.right))

def rotate_right(y):
    x = y.left
    y.left = x.right
    x.right = y
    update_height(y)
    update_height(x)
    return x

def rotate_left(x):
    y = x.right
    x.right = y.left
    y.left = x
    update_height(x)
    update_height(y)
    return y

def avl_insert(node, val):
    if node is None:
        return AVLNode(val)
    if val < node.val:
        node.left = avl_insert(node.left, val)
    else:
        node.right = avl_insert(node.right, val)

    update_height(node)
    balance = balance_factor(node)

    if balance > 1 and val < node.left.val:           # left-left
        return rotate_right(node)
    if balance < -1 and val > node.right.val:         # right-right
        return rotate_left(node)
    if balance > 1 and val > node.left.val:           # left-right
        node.left = rotate_left(node.left)
        return rotate_right(node)
    if balance < -1 and val < node.right.val:         # right-left
        node.right = rotate_right(node.right)
        return rotate_left(node)
    return node

root = None
for v in [10, 20, 30, 40, 50, 25]:                 # would degrade a plain BST
    root = avl_insert(root, v)
print(f"Balanced height: {height(root)}")          # 3, not 6
```

⚠ When NOT to Use

More rotation overhead than Red-Black trees on **frequent insertions/deletions** — if reads vastly outnumber writes, AVL's stricter balance (faster lookups) is worth it; if writes are frequent, Red-Black trees are often preferred.

★ Red-Black Tree — *Self-Balancing BST (Relaxed Balance)*

🔧 How It Works

Each node is colored **red or black**, with rules that prevent any path from root to leaf from being more than twice as long as any other (rules: root is black, red nodes can't have red children, every path has the same number of black nodes). This is a looser balance than AVL, requiring **fewer rotations** on average.

🕒 Complexity

Operation	All cases (guaranteed)
Search / Insert / Delete	$O(\log n)$

🌐 Real-World Use

This is the structure behind **Java's TreeMap/TreeSet**, **C++'s `std::map/std::set`**, and the **Linux kernel's process scheduler (CFS)** — chosen over AVL specifically because insert/delete-heavy workloads benefit from fewer rebalancing operations.

📄 Code

```
# Full production Red-Black trees are intricate (~200+ lines with
all rotation
# cases). Here is the core insight – color rules and why they bound
height –
# shown via the invariant checker, which is what you'd actually
verify:

class RBNode:
    def __init__(self, val, color="RED"):
        self.val = val
        self.color = color          # "RED" or "BLACK"
        self.left = None
        self.right = None

def black_height_valid(node):
    """Every path from a node to its descendant leaves has the same
    number of black nodes – this invariant is what bounds the tree
    height
    to  $O(\log n)$  even without AVL's strict balance factor."""
    if node is None:
        return 1 # null leaves count as black
    left_bh = black_height_valid(node.left)
    right_bh = black_height_valid(node.right)
    if left_bh != right_bh or left_bh == 0:
        return 0 # invalid
    return left_bh + (1 if node.color == "BLACK" else 0)

# In practice, use a library implementation (e.g., Python's
'sortedcontainers'
# module uses balanced trees internally) rather than hand-rolling
one in production.
from sortedcontainers import SortedList
```

```
sl = SortedList([5, 3, 7, 1])
sl.add(4)
print(list(sl)) # [1, 3, 4, 5, 7] - O(log n) insert, backed by
balanced structure
```

⚠ When NOT to Use

Implementing one from scratch is complex and error-prone — in real projects, use a well-tested library (sortedcontainers in Python, TreeMap in Java) rather than hand-rolling rotation logic.

★ Segment Tree — *Range Query Structure*

🔧 How It Works

A binary tree where each node represents an **aggregate** (sum, min, max) over a range of the array. Leaves are individual elements; each internal node combines its two children. Supports range queries and point updates in $O(\log n)$.

⚙ Complexity

Operation	Time	Space
Build	$O(n)$	$O(n)$
Range Query	$O(\log n)$	—
Point Update	$O(\log n)$	—

🌐 Real-World Use

Real-time analytics dashboards (running range sums/min/max over time-series data), competitive programming range-query problems, and interval-based scheduling conflict checks.

📄 Code

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0] * (2 * self.n)
        for i in range(self.n):
            self.tree[self.n + i] = data[i]
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def update(self, index, value):
        i = index + self.n
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]

    def range_sum(self, left, right): # inclusive left, exclusive
right
        result = 0
        left += self.n
        right += self.n
        while left < right:
            if left % 2 == 1:
                result += self.tree[left]
                left += 1
            if right % 2 == 1:
                right -= 1
            left //= 2
            right //= 2
```

```

        result += self.tree[right]
        left //= 2
        right //= 2
    return result

st = SegmentTree([1, 3, 5, 7, 9, 11])
print(st.range_sum(1, 4))    # 3+5+7 = 15
st.update(1, 10)             # change index 1 from 3 to 10
print(st.range_sum(1, 4))    # 10+5+7 = 22

```

⚠ When NOT to Use

Overkill if the array is **static and you never need range queries** (a simple prefix-sum array is $O(1)$ query, $O(n)$ space, no updates needed). Segment trees earn their complexity when you need both **updates and range queries**.

★ Fenwick Tree (Binary Indexed Tree) — Compact Range Sum Structure

🔧 How It Works

A more memory-efficient alternative to segment trees for **prefix sums**, using clever bit manipulation (i & $-i$ to find the lowest set bit) to know which indices a node is responsible for.

🕒 Complexity

Operation	Time	Space
Build	$O(n \log n)$	$O(n)$
Prefix Sum Query	$O(\log n)$	—
Point Update	$O(\log n)$	—

🌐 Real-World Use

Counting inversions in an array (used in measuring “how sorted” data is), cumulative frequency tables in statistics, and leaderboard ranking systems needing fast “how many players scored below X” queries.

📦 Code

```

class FenwickTree:
    def __init__(self, size):
        self.n = size
        self.tree = [0] * (size + 1)

    def update(self, index, delta): # index is 0-based
        i = index + 1
        while i <= self.n:
            self.tree[i] += delta
            i += i & (-i)           # move to next responsible
node

    def prefix_sum(self, index):    # sum of elements [0, index]
        i = index + 1
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & (-i)           # move to parent range
        return total

```

```

        def range_sum(self, left, right): # inclusive both ends
            return self.prefix_sum(right) - (self.prefix_sum(left - 1)
if left > 0 else 0)

ft = FenwickTree(6)
for i, val in enumerate([1, 3, 5, 7, 9, 11]):
    ft.update(i, val)
print(ft.range_sum(1, 3)) # 3+5+7 = 15

```

⚠ When NOT to Use

Only naturally supports **prefix-sum-style** aggregates (sum, XOR) — for min/max range queries, a segment tree is required since Fenwick trees can't "undo" a min/max the way they can undo a sum.

★ Trie (Prefix Tree) — String Prefix Structure

🔧 How It Works

A tree where each path from root to a node represents a **prefix**, and each edge represents one character. Words sharing a prefix share the same path, making prefix lookups extremely fast.

⌚ Complexity

Operation	Time	Space
Insert	O(L)	O(L) per word
Search / Prefix Search	O(L)	—

L = length of the word/prefix.

🌐 Real-World Use

Autocomplete and search suggestions (Google search bar, IDE code completion), spell checkers, IP routing tables (longest prefix match), and T9 predictive text on old phones.

💻 Code

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_end_of_word = True

    def search(self, word):
        node = self._find_node(word)
        return node is not None and node.is_end_of_word

    def starts_with(self, prefix):
        return self._find_node(prefix) is not None

```

```

def _find_node(self, prefix):
    node = self.root
    for ch in prefix:
        if ch not in node.children:
            return None
        node = node.children[ch]
    return node

trie = Trie()
for word in ["cat", "car", "card", "care", "dog"]:
    trie.insert(word)
print(trie.search("car"))           # True
print(trie.search("ca"))            # False (not a complete word)
print(trie.starts_with("ca"))       # True

```

⚠ When NOT to Use

Can use significant memory for large alphabets or sparse datasets (each node stores a dictionary of children) — compressed variants (radix trees / Patricia tries) are used in memory-sensitive contexts like IP routing tables.

★ Union-Find / Disjoint Set Union (DSU) — Grouping Structure

🔧 How It Works

Tracks a collection of disjoint (non-overlapping) sets, supporting two operations: `find(x)` (which set is `x` in?) and `union(x, y)` (merge `x`'s and `y`'s sets). **Path compression** (flatten the tree during find) and **union by rank** (attach smaller tree under larger) make both operations nearly $O(1)$.

⌚ Complexity

Operation	Time (with both optimizations)
Find / Union	$O(\alpha(n))$ — effectively constant

α = inverse Ackermann function, grows so slowly it's practically ≤ 4 for any real-world n .

🌐 Real-World Use

Kruskal's MST algorithm (cycle detection, shown earlier), detecting connected components in social networks ("are these two people in the same friend group?"), image processing (connected-component labeling for blob detection), and network connectivity checks.

💻 Code

```

class UnionFind:
    def __init__(self, elements):
        self.parent = {e: e for e in elements}
        self.rank = {e: 0 for e in elements}

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path
compression
        return self.parent[x]

```

```

def union(self, x, y):
    root_x, root_y = self.find(x), self.find(y)
    if root_x == root_y:
        return False # already connected
    if self.rank[root_x] < self.rank[root_y]: # union by
rank
        root_x, root_y = root_y, root_x
    self.parent[root_y] = root_x
    if self.rank[root_x] == self.rank[root_y]:
        self.rank[root_x] += 1
    return True

uf = UnionFind(["A", "B", "C", "D", "E"])
uf.union("A", "B")
uf.union("C", "D")
print(uf.find("A") == uf.find("B")) # True
print(uf.find("A") == uf.find("C")) # False
uf.union("B", "C")
print(uf.find("A") == uf.find("D")) # True - now transitively
connected

```

⚠ When NOT to Use

Doesn't support **splitting** sets apart once merged — if your problem requires undoing a union, you need a different structure (or a “Union-Find with rollback” variant using a different implementation strategy).

7 Special & Modern Topics

★ Backtracking: N-Queens — *Constraint Satisfaction*

⚙ How It Works

Place queens on a chessboard one row at a time. Before placing a queen, check it doesn't conflict with previously placed queens (same column or diagonal). If a placement leads to a dead end, **backtrack** — undo the last placement and try the next option.

⌚ Complexity

Case	Time	Space
Worst case	$O(n!)$	$O(n)$

Pruning (checking constraints early) makes it far faster in practice than brute force $O(n^n)$.

🌐 Real-World Use

Constraint satisfaction problems broadly: register allocation in compilers, resource allocation with conflicting constraints, and any “try, check, undo if invalid” search space (VLSI chip design, exam timetable scheduling).

💻 Code

```

def solve_n_queens(n):
    solutions = []
    columns = set()
    diagonals = set() # row - col is constant on a diagonal

```

```

    anti_diagonals = set() # row + col is constant on the other
    diagonal
    board = [-1] * n      # board[row] = column of queen in that
    row

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return
        for col in range(n):
            if col in columns or (row - col) in diagonals or (row +
col) in anti_diagonals:
                continue # conflict - skip
            columns.add(col)
            diagonals.add(row - col)
            anti_diagonals.add(row + col)
            board[row] = col

            backtrack(row + 1) # recurse

            columns.remove(col) # backtrack: undo the
choice
            diagonals.remove(row - col)
            anti_diagonals.remove(row + col)

        backtrack(0)
        return solutions

    solutions = solve_n_queens(4)
    print(f"Found {len(solutions)} solutions") # 2
    print(solutions[0]) # e.g. [1, 3, 0, 2]

```

⚠ When NOT to Use

For large n (say, > 20), even pruned backtracking becomes slow — specialized heuristics or approximation methods take over for very large constraint problems.

★ Backtracking: Sudoku Solver — Constraint Satisfaction

🔍 How It Works

Find an empty cell, try each digit 1-9, check row/column/3×3-box validity, recurse into the next empty cell, and backtrack if no digit leads to a solution.

⌚ Complexity

Case	Time	Space
Worst case	$O(9^{(\text{empty cells})})$	$O(1)$ extra (in-place)

In practice, constraint checking prunes this dramatically.

🌐 Real-World Use

Same family as N-Queens — general **constraint propagation** logic used in AI planning systems, automated theorem proving, and puzzle-generation/validation software.

📄 Code

```

def solve_sudoku(board):
    empty = _find_empty(board)
    if not empty:
        return True # solved
    row, col = empty

    for num in "123456789":
        if _is_valid(board, row, col, num):
            board[row][col] = num
            if solve_sudoku(board):
                return True
            board[row][col] = "." # backtrack
    return False

def _find_empty(board):
    for r in range(9):
        for c in range(9):
            if board[r][c] == ".":
                return (r, c)
    return None

def _is_valid(board, row, col, num):
    if num in board[row]:
        return False
    if num in [board[r][col] for r in range(9)]:
        return False
    box_row, box_col = 3 * (row // 3), 3 * (col // 3)
    for r in range(box_row, box_row + 3):
        for c in range(box_col, box_col + 3):
            if board[r][c] == num:
                return False
    return True

# '.' represents an empty cell
board = [
    list("53..7...."), list("6..195..."), list(".98...6."),
    list("8...6...3"), list("4..8.3..1"), list("7...2...6"),
    list(".6....28."), list("...419..5"), list("....8..79"),
]
solve_sudoku(board)
for row in board:
    print("".join(row))

```

⚠ When NOT to Use

Some real-world Sudoku puzzles are designed to be brutal for backtracking (very few clues) — production solvers add heuristics like “most constrained cell first” (fewest valid options) to prune the search space much faster.

★ Bit Manipulation: Fast Exponentiation — *Divide and Conquer*

🔧 How It Works

Instead of multiplying base by itself exp times ($O(\text{exp})$), use the fact that $\text{base}^{\text{exp}} = (\text{base}^{(\text{exp}/2)})^2$ when exp is even, recursively halving the exponent — $O(\log \text{exp})$ multiplications.

⌚ Complexity

Case	Time	Space
All cases	$O(\log n)$	$O(\log n)$ recursive, $O(1)$ iterative

🌐 Real-World Use

Cryptography (RSA encryption relies on fast modular exponentiation of huge numbers), computing large Fibonacci numbers via matrix exponentiation, and any performance-critical power computation.

🔗 Code

```
def fast_power(base, exp, mod=None):
    result = 1
    base = base % mod if mod else base
    while exp > 0:
        if exp % 2 == 1:           # if current bit is 1, include
this base power
            result = (result * base) % mod if mod else result * base
        exp //= 2
        base = (base * base) % mod if mod else base * base
    return result

print(fast_power(2, 10))          # 1024
print(fast_power(7, 128, mod=13)) # modular exponentiation (used
in RSA)
```

⚠ When NOT to Use

For tiny, fixed exponents (like squaring, x^2), just use direct multiplication — the log-time win only matters for large or variable exponents.

★ Bit Manipulation: Single Number — XOR Trick

🔍 How It Works

Given an array where every element appears **twice except one**, XOR-ing all elements together cancels out every pair (since $x \oplus x = 0$), leaving only the single unpaired number ($x \oplus 0 = x$).

⌚ Complexity

Case	Time	Space
All cases	$O(n)$	$O(1)$

🌐 Real-World Use

Checksums and error detection (XOR parity checks in networking/storage — RAID uses XOR to reconstruct lost data), and it's a favorite in low-level embedded programming where $O(1)$ space matters.

🔗 Code

```
def single_number(nums):
    result = 0
    for n in nums:
        result ^= n
    return result

print(single_number([4, 1, 2, 1, 2])) # 4
```

⚠ When NOT to Use

Only works cleanly for the “everything appears twice except one” pattern — variants (everything appears **three** times except one) need a different bit-counting trick, not plain XOR.

★ Reservoir Sampling — *Randomized Algorithm*

🔧 How It Works

Select k random items from a stream of **unknown or unbounded length**, with each item having an equal probability of being chosen, using only $O(k)$ memory. Fill the reservoir with the first k items, then for each subsequent item i , replace a random reservoir slot with probability k/i .

🕒 Complexity

Case	Time	Space
All cases	$O(n)$	$O(k)$

🌐 Real-World Use

Sampling from live data streams — selecting a random sample of log lines from an infinite log stream, A/B test sampling from live user traffic, and random tweet sampling from Twitter’s firehose without loading the entire stream into memory.

🔗 Code

```
import random

def reservoir_sample(stream, k):
    reservoir = []
    for i, item in enumerate(stream):
        if i < k:
            reservoir.append(item)
        else:
            j = random.randint(0, i)    # random index in [0, i]
            if j < k:
                reservoir[j] = item
    return reservoir

stream = range(1, 10001)    # simulating a stream of 10,000 items
sample = reservoir_sample(stream, k=5)
print(sample)    # 5 uniformly random items from the stream
```

⚠ When NOT to Use

If the total size of the stream is known in advance and fits in memory, simpler random sampling (`random.sample()`) is more straightforward — reservoir sampling’s value is specifically for **unknown-length or infinite streams**.

★ Bloom Filter — *Probabilistic Data Structure*

🔧 How It Works

A space-efficient structure that answers “is this item **possibly** in the set?” using a bit array and multiple hash functions. It can have **false positives** (says “yes” when it’s actually not there) but **never false negatives** (if it says “no,” the item is definitely not there).

⚙️ Complexity

Operation	Time	Space
Insert / Check	$O(k)$ — k hash functions	$O(m)$ — m bits, independent of item count

🌍 Real-World Use

Checking if a username is already taken before hitting the database, web browsers checking URLs against a malicious-site blacklist (Chrome’s Safe Browsing), and databases (Cassandra, HBase) using Bloom filters to avoid unnecessary disk reads for keys that don’t exist.

💻 Code

```
import hashlib

class BloomFilter:
    def __init__(self, size=1000, num_hashes=3):
        self.size = size
        self.num_hashes = num_hashes
        self.bits = [0] * size

    def _hashes(self, item):
        for i in range(self.num_hashes):
            digest = hashlib.md5(f"{item}{i}".encode()).hexdigest()
            yield int(digest, 16) % self.size

    def add(self, item):
        for h in self._hashes(item):
            self.bits[h] = 1

    def might_contain(self, item):
        return all(self.bits[h] for h in self._hashes(item))

bf = BloomFilter(size=1000, num_hashes=3)
bf.add("alice")
bf.add("bob")
print(bf.might_contain("alice"))    # True (definitely added)
print(bf.might_contain("charlie"))  # False (definitely not added) —
usually                               # can occasionally be a false
positive
```

⚠️ Edge Cases & Pitfalls

- **Cannot delete items** from a standard Bloom filter (clearing a bit might break another item’s membership) — a “Counting Bloom Filter” variant is needed if deletion is required.
- False positive rate rises as more items are added relative to the bit array size — size the filter appropriately for the expected number of insertions, or use a formula to compute optimal m and k for your target false-positive rate.

📄 Quick Reference: Complexity Cheat Sheet

Algorithm	Time (Average)	Space
Bubble / Selection / Insertion Sort	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(1)$
Counting / Radix Sort	$O(n + k)$	$O(n + k)$
Binary Search	$O(\log n)$	$O(1)$
KMP / Rabin-Karp	$O(n + m)$	$O(m)$
BFS / DFS	$O(V + E)$	$O(V)$
Dijkstra's	$O((V+E) \log V)$	$O(V)$
Bellman-Ford	$O(V \times E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal's / Prim's	$O(E \log V)$	$O(V)$
0/1 Knapsack / LCS / Edit Distance	$O(n \times m)$	$O(n \times m)$
Kadane's Algorithm	$O(n)$	$O(1)$
BST (balanced)	$O(\log n)$	$O(n)$
Trie Search	$O(L)$	$O(\text{alphabet} \times \text{nodes})$
Union-Find	$O(\alpha(n)) \approx O(1)$	$O(n)$

✂ **Final takeaway:** The goal isn't to memorize every implementation — it's to recognize the **pattern** a problem is hinting at (sorted data → binary search; overlapping subproblems → DP; "shortest path" → BFS/Dijkstra; "connected groups" → Union-Find/DFS) and know which tool fits.